Multimodal Anti-Money Laundering Model Lifecycle Management

MLOps Class Project Proposal

Data2Deploy
Jaya Prakash Yadav Gorla | Neha Anusooya Thimmarayi | Preshita Soni | Rajani Meka

GraphSAGE Transaction graph | **DistilBERT** Payment memo text | **BiLSTM** Behavioral time-series

1.1 Project Scope and Objectives

Money laundering — the process of disguising illegally obtained funds as legitimate income — costs the global economy an estimated $800 billion to $2 trillion annually (UNODC, 2023). Traditional rule-based AML systems generate up to 95% false-positive alerts, overwhelming compliance teams while sophisticated laundering schemes slip through undetected. The core problem is that transaction graph topology, temporal behavioral patterns, and payment description text each carry complementary signals that siloed detectors cannot exploit.

This project builds a multimodal ML system that fuses all three signals into a single late-fusion neural network and wraps it in a complete MLOps lifecycle — from data versioning and experiment tracking through CI/CD-gated deployment and continuous drift monitoring. The regulatory angle (SHAP explainability, model cards, drift auditing) mirrors what financial institutions must satisfy under Basel IV and FinCEN guidance, making this directly applicable to production AI governance.

Objectives

- Train a multimodal AML detector fusing transaction graph structure (GraphSAGE), payment memo text (DistilBERT), and behavioral time-series (BiLSTM) via a late-fusion MLP.
- Demonstrate fusion outperforms each single-modality baseline on AUC-PR (primary metric) with a target of ≥ 0.80.
- Build and operate a full MLOps pipeline: DVC data versioning, MLflow experiment tracking, GitHub Actions CI/CD with an evaluation gate, and AWS SageMaker deployment.
- Satisfy regulatory explainability requirements using SHAP force plots per prediction and a Google Model Card.

Success metrics

Metric	Target	Rationale
AUC-PR (primary)	≥ 0.80	Robust to ~2% illicit class imbalance
Precision @ Recall = 0.8	≥ 0.70	Regulatory: catch 80% of fraud cases
False positive rate	≤ 5%	Compliance teams cannot review more than 5% of volume
Inference latency (P95)	< 200 ms	Real-time transaction screening SLA
Fusion > each branch alone	Required	Validates multimodal fusion adds value

1.2 Selection of Data

All datasets are publicly available and require no licensing agreements. They were selected to provide a realistic and varied representation of AML transaction patterns across all three modalities.

Dataset	Modality	Contents	Preprocessing
Elliptic Bitcoin Dataset (Kaggle)	Graph	203K transactions, 234K edges, 166 features, 2% illicit labels	Build NetworkX graph → PyG HeteroData; normalize node features; SMOTE for class balance
PaySim (Kaggle)	Time-series	6.3M synthetic mobile-money transactions with timestamp, type, amount, account IDs	30-day rolling windows per account; extract velocity, hour-of-day, day-of-week features
Synthetic memo text (generated)	Text (NLP)	~50K payment descriptions generated from domain templates (consulting, wire, invoice, charity)	DistilBERT tokenizer; max length 64 tokens; zero-vector for missing memo fields

Class imbalance: The Elliptic dataset contains ~2% illicit transactions. We will apply SMOTE oversampling to the tabular branch and focal loss ($\gamma = 2$) to the GNN to handle this. AUC-PR is used as the primary metric rather than ROC-AUC, which is misleading under severe imbalance.

1.3 Model Considerations

The system uses a three-branch late-fusion architecture. Each modality is encoded independently by a specialized model; the resulting embeddings are concatenated and passed through a shared MLP fusion head that outputs a calibrated AML risk score in [0, 1].

Branch 1 — Transaction graph: GraphSAGE (PyTorch Geometric)

Architecture: 2-layer GraphSAGE (Hamilton et al., 2017) implemented in PyTorch Geometric. Inductive sampling avoids full-graph neighborhood aggregation, keeping training under 2 hours on a free Kaggle GPU. Output: 128-dimensional node embedding per account. Loss: focal loss to handle class imbalance.

Why GraphSAGE over GCN: GraphSAGE is inductive — it generalizes to unseen nodes at inference time without retraining, which is essential for a production AML system encountering new accounts daily.

Branch 2 — Payment memo text: DistilBERT (HuggingFace Transformers)

Architecture: DistilBERT (Sanh et al., 2019) fine-tuned for 3 epochs on synthetic memo text. The [CLS] embedding (64-dim projection) captures domain-specific patterns such as round-number amounts, vague counterparty descriptions, and high-frequency transfer language. Optimizer: AdamW, lr = 2e−5.

Branch 3 — Behavioral time-series: Bidirectional LSTM

Architecture: 2-layer BiLSTM on 30-day rolling windows of [amount, hour-of-day, day-of-week, transaction type, cumulative velocity]. Output: 64-dimensional hidden state. Trains in ~15 minutes on CPU.

Fusion head

Architecture: MLP with layers [256 → 128 → 64 → 1], ReLU activations, dropout 0.3, sigmoid output. Input is the concatenation of all three branch embeddings [128 + 64 + 64]. Platt scaling is applied post-training for calibrated probability estimates.

Baseline: XGBoost trained on handcrafted tabular features (amount, velocity, graph degree, n-hop neighbor count) serves as the benchmark. All three branches and the fusion model must outperform it on AUC-PR.

1.4 Open-Source Tools

The project uses two third-party frameworks from the PyTorch ecosystem as its core open-source tools, alongside a full MLOps stack. All tools are open-source, actively maintained, and compatible with each other.

Tool	Role	Integration in this project
PyTorch Geometric (PyG)	Third-party framework ✓	Builds the transaction graph and trains the GraphSAGE model. Directly listed as a recommended third-party tool in the course guidelines.
HuggingFace Transformers	Third-party framework ✓	Fine-tunes DistilBERT on payment memo text. Also listed in course guidelines. Compatible with PyTorch training loop.
MLflow	Experiment tracking	Logs all runs (params, metrics, artifacts); hosts model registry for staging → production promotion.
DVC	Data versioning	Versions Elliptic + PaySim datasets and trained model artifacts in S3/Google Drive.
GitHub Actions	CI/CD	On every PR: lint → unit tests → data schema checks → training smoke test → AUC-PR evaluation gate → Docker build → deploy.
Great Expectations	Data quality	Validates schema, null rates, and class balance at ingestion; gates the CI pipeline.
Evidently AI	Drift monitoring	Daily drift reports per modality (PSI for graph features, KS test for time-series, cosine drift for text embeddings).
AWS SageMaker	Deployment	Hosts the containerized (Docker + BentoML) inference endpoint with canary rollout and autoscaling.
SHAP	Explainability	Generates per-prediction force plots across all three modalities for regulatory compliance.

Team Split and 5-Week Timeline

Week	Focus	Tasks by member
1	Data + setup	A: Elliptic graph construction (NetworkX → PyG), EDA B: PaySim behavioral windows, synthetic memo generation C: DVC + MLflow setup, Great Expectations suite, repo structure D: AWS account, Docker base image, Codespaces devcontainer.json
2	Model training	A: GraphSAGE training (Kaggle GPU, focal loss), log to MLflow B: DistilBERT fine-tune (Kaggle GPU) + BiLSTM on CPU, log to MLflow C: GitHub Actions CI YAML, unit tests, data schema gate D: BentoML service scaffold, FastAPI /predict stub
3	Fusion + eval	A+B: Late fusion MLP, Platt calibration, SHAP force plots A+B: Ablation study (each branch vs fusion), model card draft C: AUC-PR evaluation gate in CI, MLflow model promotion logic D: SageMaker endpoint config, canary rollout YAML, Evidently wired
4	CI/CD + deploy	A+B: Integration testing, latency benchmarking, fix regressions C: Full CI/CD pipeline live, Docker push to ECR, SageMaker deploy via CI D: Grafana dashboard live, Airflow retrain DAG, canary 10% traffic test
5	Polish + demo	A: Final ablation report, README + architecture diagram B: Demo video recording, model card finalized C: Final CI/CD validation, proposal → final report D: Live dashboard demo, drift simulation test, presentation slides

References

[1] Hamilton, W., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs. NeurIPS. — GraphSAGE architecture used in the graph branch.

[2] Sanh, V., et al. (2019). DistilBERT, a distilled version of BERT. EMC2 Workshop, NeurIPS. — Foundation for the text branch model.

[3] Weber, M., et al. (2019). Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks. KDD Workshop. — Introduces the Elliptic dataset.

[4] Lopez-Rojas, E. A., et al. (2016). PaySim: A financial mobile money simulator for fraud detection. EMSS. — Source of the PaySim dataset.

[5] Lin, T.-Y., et al. (2017). Focal loss for dense object detection. ICCV. — Focal loss for GNN class imbalance.

[6] Lundberg, S., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. NeurIPS. — SHAP explainability framework.

[7] Mitchell, M., et al. (2019). Model cards for model reporting. FAccT. — Standard followed for regulatory model documentation.

[8] Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. ICLR Workshop. — PyG library (third-party framework).